Design & Implementation of Logic Gates using BC547 Transistors

EXPLORE MORE PROJECTS

SCAN ME

DEVANSH UMAR
Electrical & Instrumentation Eng. Student

From an Exam Question to Actual Hardware

Learning & Simulation Report Prepared By:

Devansh Umar [B. Tech (EIE) 5th Sem]

Coursework of – **Microprocessors and Interfacing**

Key Points of this Project -

Why I Started This 2
What I Wanted to Find Out 2
The Basic Idea 3
My Approach 4
First Step - NOT Gate 4
NOR Gate - Where It Started Getting Interesting 6
OR Gate - NOR + NOT 7
NAND and AND 9
XOR - The Part That Made Me Laugh 11
XNOR 13
The Seven Gates 14
Simulation vs Real Circuit 14
What This Project Actually Taught Me 15
The Bigger Connection 15
Conclusion 16

Why I Started This

This project didn't start with me planning to make a logic-gate project.

It actually started with an exam question.

Back in my **4th semester**, we had a question in -

Microprocessors and Interfacing: Design AND, NAND, OR, RAM and ROM using discrete components.

The funny part was that this question wasn't new. It appeared in our **internal examination**, **external examination**, and even in an **older question paper.**

So obviously, by the time I saw it again, I thought I knew what was expected.

Except... I didn't.

In the internal examination, I made the mistake of designing the gates using their **logic symbols** instead of actually implementing those using **discrete components**.

My teacher caught the mistake and deducted marks.

By the external examination, I had understood what the question actually meant, and I implemented the gates using discrete components correctly.

I could have just left the story there.

But later, when I had a few **BC547 transistors** with me, the same question came back to my mind.

And then came the thought -

"If I already know how the gate is supposed to work, why not actually build one?"

That was basically the beginning of this project.

What I Wanted to Find Out

I had already studied logic gates theoretically.

I knew -

AND	A.B
OR	A + B
XNOR	AB + A`B`
XOR	AB` + A`B

and so on.

But knowing the equation and actually seeing the transistor produce that logic are two different things.

So I wanted to find out -

- How can a BC547 actually behave like a logic switch?
- How can multiple transistors be combined to make a gate?
- Can I reproduce the same logic in LTspice?
- Will the simulated circuit actually behave like the physical circuit?
- What happens when the circuit doesn't give the output I expected?
- Can I build the complete set of basic gates without using ready-made logic ICs?

That became the real purpose of the project.

The Basic Idea

Before making the gates, I first needed to understand one simple thing -

What happens when I use a **BC547 as a switch?**

For an **NPN transistor**, when the **base is LOW, the transistor is OFF**.

Input LOW

↓

BC547 OFF

↓

Collector is pulled HIGH

When the base is HIGH -

Input HIGH

↓

BC547 ON

↓

Collector is pulled LOW

So with a resistor connected to the collector and the transistor connected to ground, I can get an inverted output.

And suddenly -

One transistor can behave like a **NOT gate**. That was the basic building block for the rest of the project.

My Approach

I didn't want to use the built-in digital gates available in simulation software.

Instead, I wanted the simulation to represent the same idea that I would build with actual components.

So the basic setup was -

- BC547B transistor
- 10 KΩ resistors
- Power supply
- LTspice for simulation

For the two-input gates, I also wanted to test all four possible combinations -

00 → 01 → 10 → 11

So instead of manually changing the inputs every time, I used **pulse sources** in LTspice.

This made the **simulation automatically** go through all four input combinations.

That was much easier than stopping the simulation and changing the inputs manually every time.

First Step - NOT Gate

I started with the simplest possible circuit.

- One BC547.
- One input.
- One output.

The idea was simple.

If the input was 0 V –

BC547 OFF

↓

Output HIGH

If the input was 5 V -

BC547 ON

↓

Output LOW

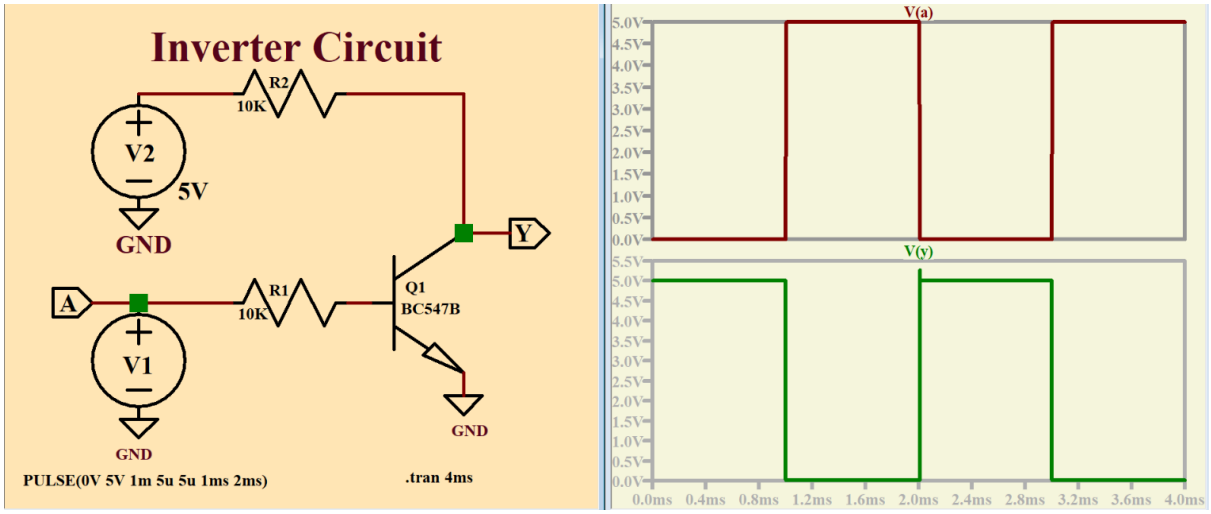
So,

Truth Table

Input (A)	Output [Y = A`]
0	1
1	0

And that's my first transistor-level logic gate.

LTspice Simulation



Physical Circuit



What I observed

- The important thing wasn't just that the waveform looked correct.
- It was that the waveform now had a direct physical explanation.
- The transistor was switching, and that switching was producing the logic inversion.

NOR Gate - Where It Started Getting Interesting

After the inverter, I moved to two inputs.

For the NOR gate, I used **two BC547** transistors.

The idea was -

- If either transistor turns ON, the output should be pulled LOW.
- Only when both transistors are OFF should the output remain HIGH.

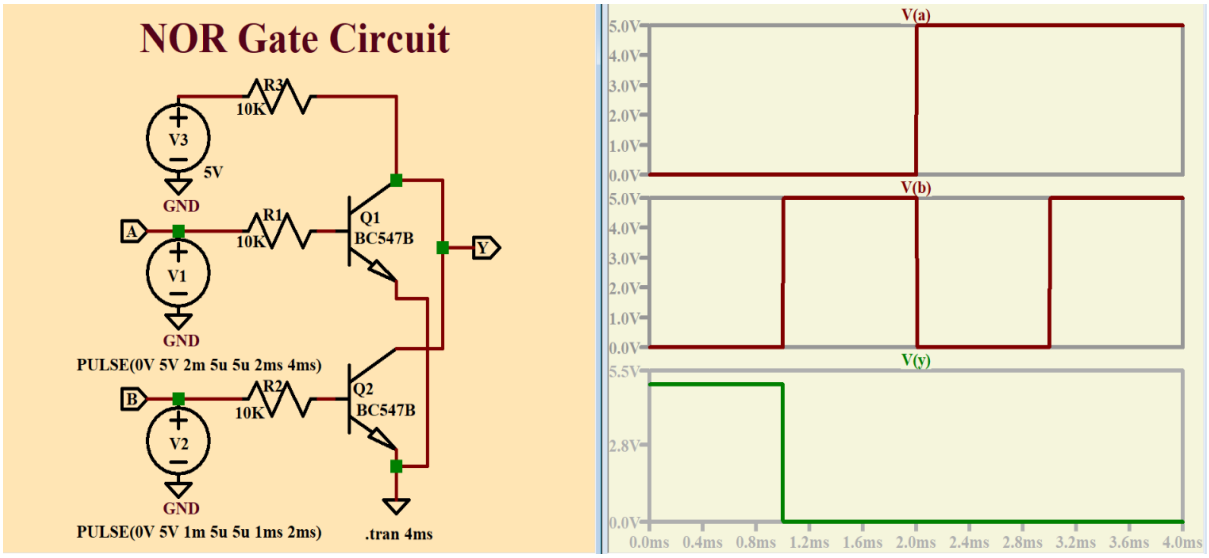
Therefore -

$$\mathbf{Y = (A+B)`}$$

Truth Table

Input (A)	Input (B)	Output [Y = (A + B)`]
0	0	1
0	1	0
1	0	0
1	1	0

LTspice Simulation



Physical Circuit



This was the point where I started seeing that the Boolean expression wasn't just something written in the notebook.

I could actually look at the circuit and ask -

Which transistor is ON right now, and where is the output current flowing?

That made the logic much easier to understand.

OR Gate - NOR + NOT

The next part was interesting because I didn't need to completely redesign everything.

I already had -

NOR = (A+B)`

So I simply inverted that output -

OR = NOR`

Therefore,

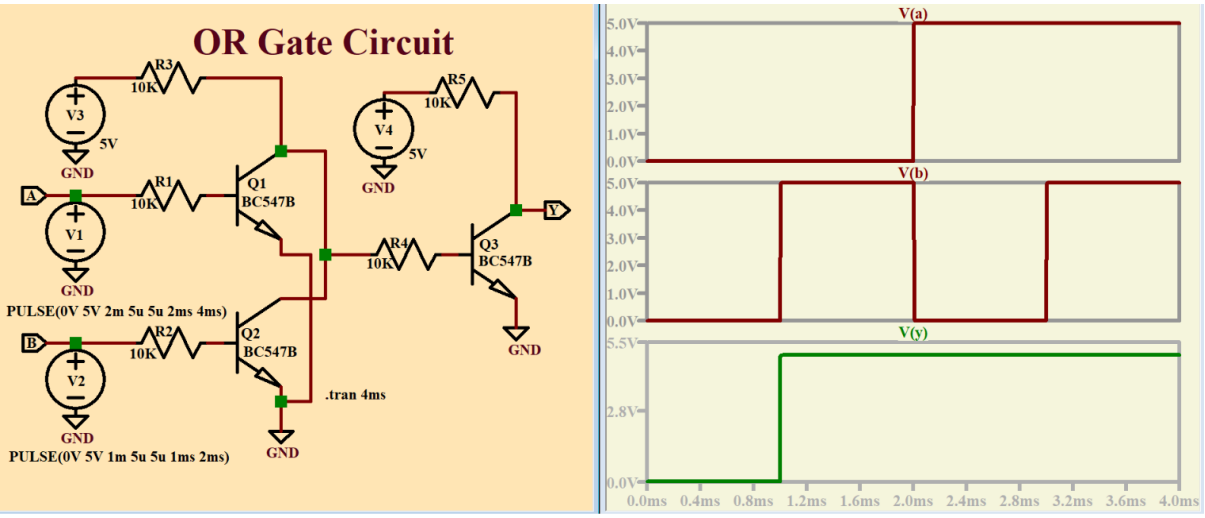
OR = A+B

This gave me the OR gate.

Truth Table

Input (A)	Input (B)	Output [Y = A + B]
0	0	0
0	1	1
1	0	1
1	1	1

LTspice Simulation



Physical Circuit



This was also a nice example of how a larger logic function can be created by combining smaller transistor stages

NAND and AND

The same idea continued with NAND and AND.

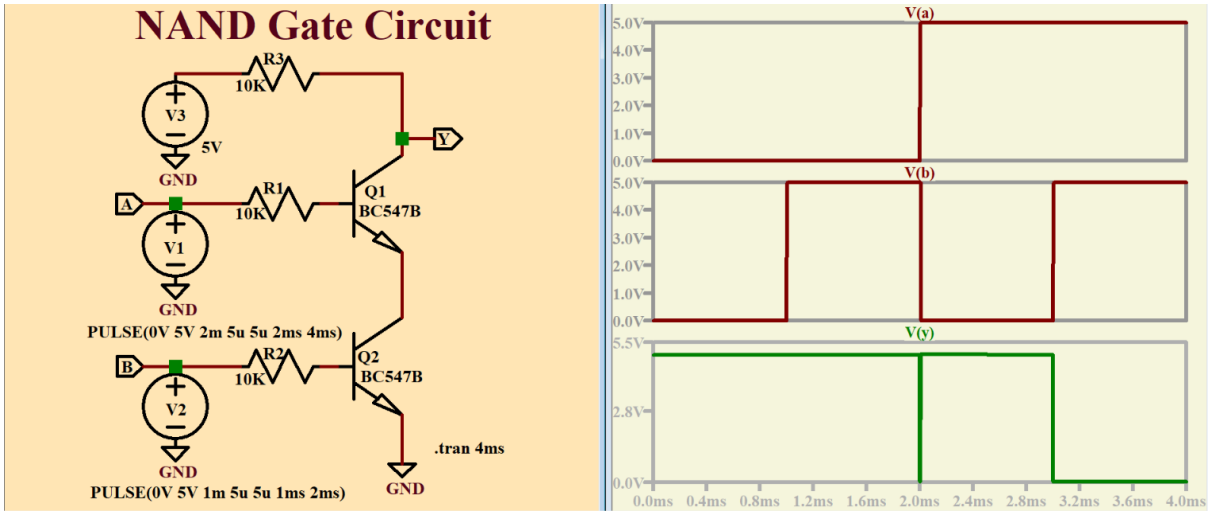
NAND is -

$$Y = (A.B)`$$

Truth Table

Input (A)	Input (B)	Output [Y = (A.B)`]
0	0	1
0	1	1
1	0	1
1	1	0

LTspice Simulation



Physical Circuit



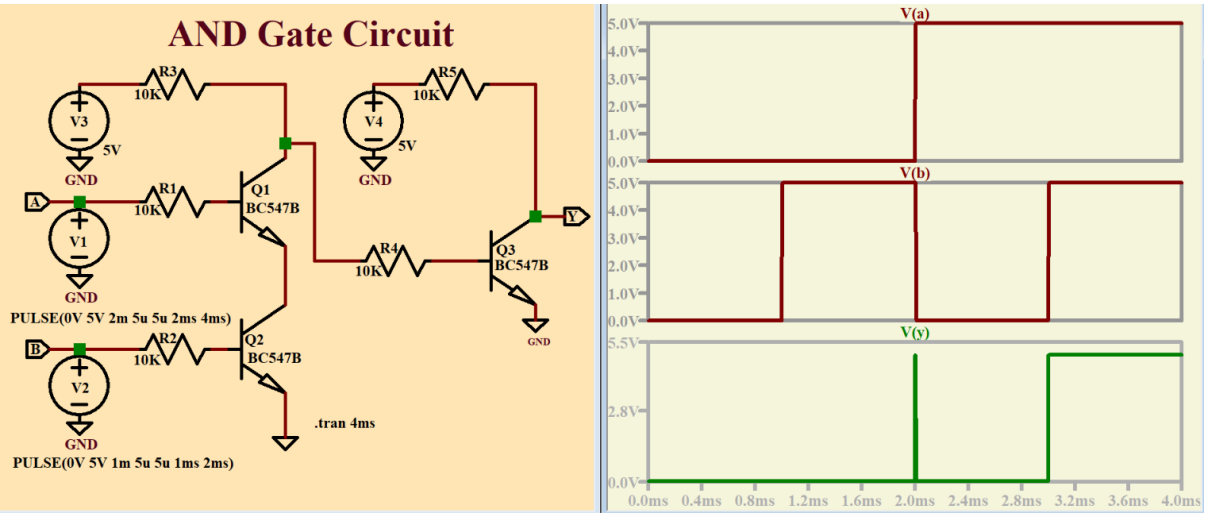
& AND is -

$$Y = A.B$$

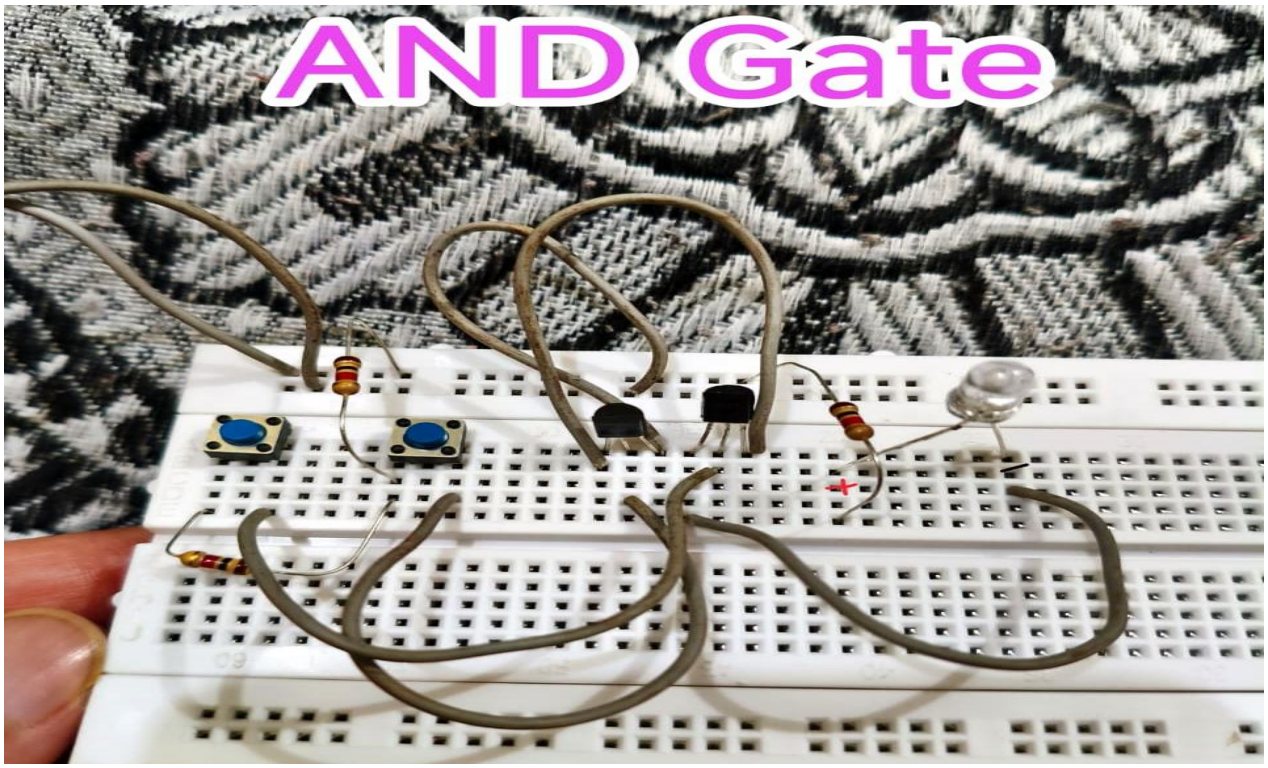
Truth Table

Input (A)	Input (B)	Output [Y = A.B]
0	0	0
0	1	0
1	0	0
1	1	1

LTspice Simulation



Physical Circuit



Here again, I checked the output using the truth table instead of just looking at whether the waveform "looked right."

XOR - The Part That Made Me Laugh

Then came XOR.

The expected truth table was -

Input (A)	Input (B)	Output [Y = A ⊗ B]
0	0	0
0	1	1
1	0	1
1	1	0

I ran the simulation.

And...

Nope.

The waveform was -



At first, obviously, I thought something had gone wrong.

Then I stopped looking at the circuit and looked at the truth table.

And immediately:

Wait... this is XNOR.

Because -

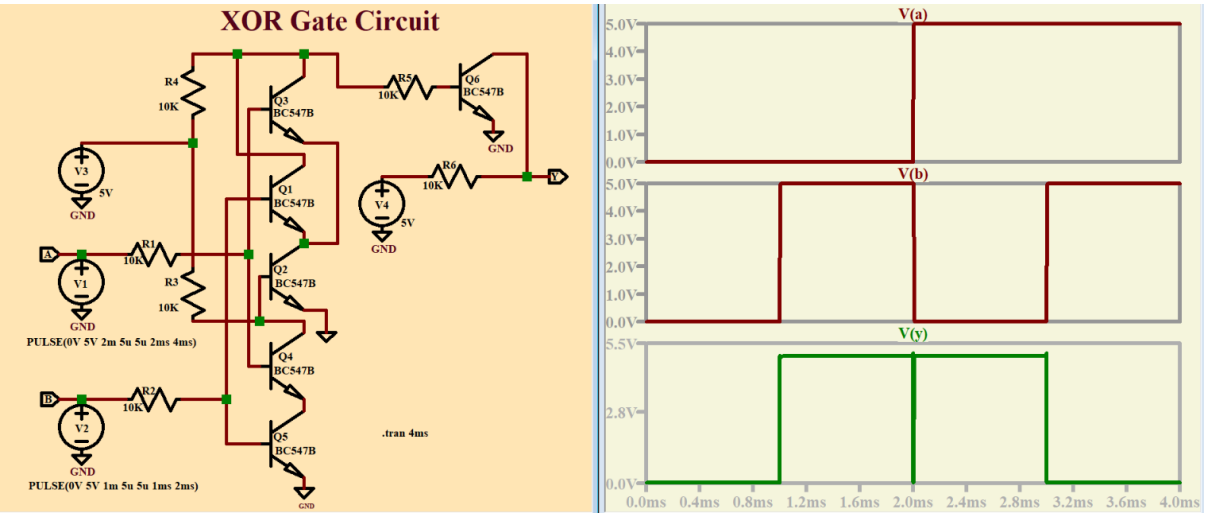
XNOR=XOR`

AB + A`B` = (AB` + A`B)`

So instead of throwing the circuit away, I treated it as a useful intermediate stage.

I added one more transistor inverter.

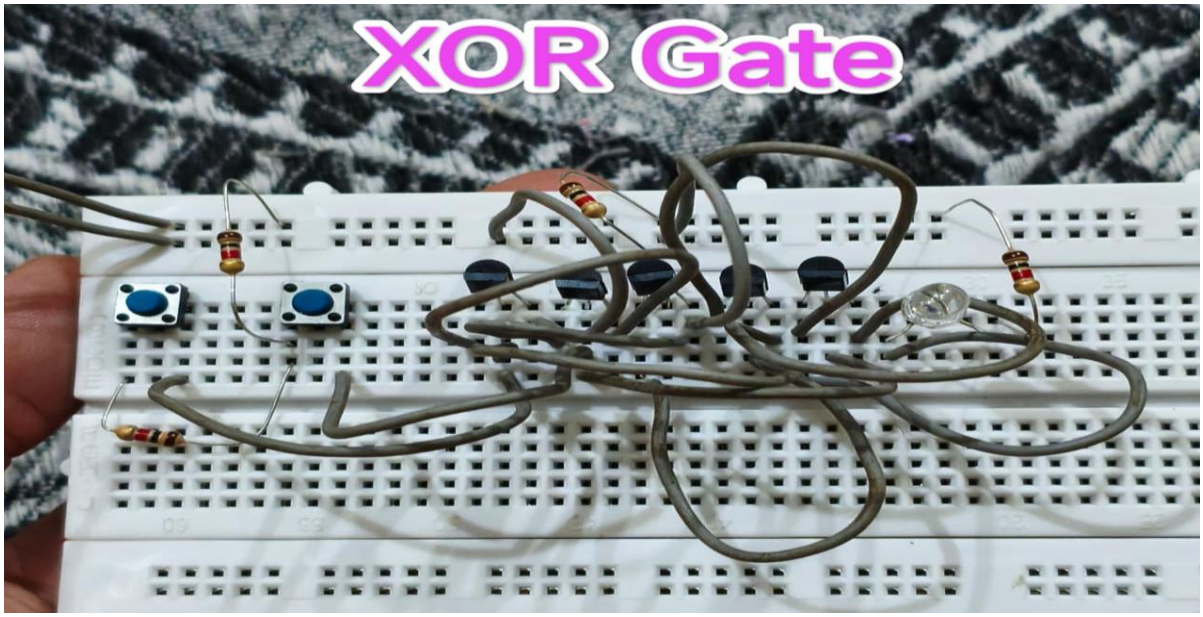
And finally –



Now it was actually XOR.

That was probably my favourite part of the simulation because the mistake itself taught me something.

Physical Circuit



XNOR

I already got this one accidently.

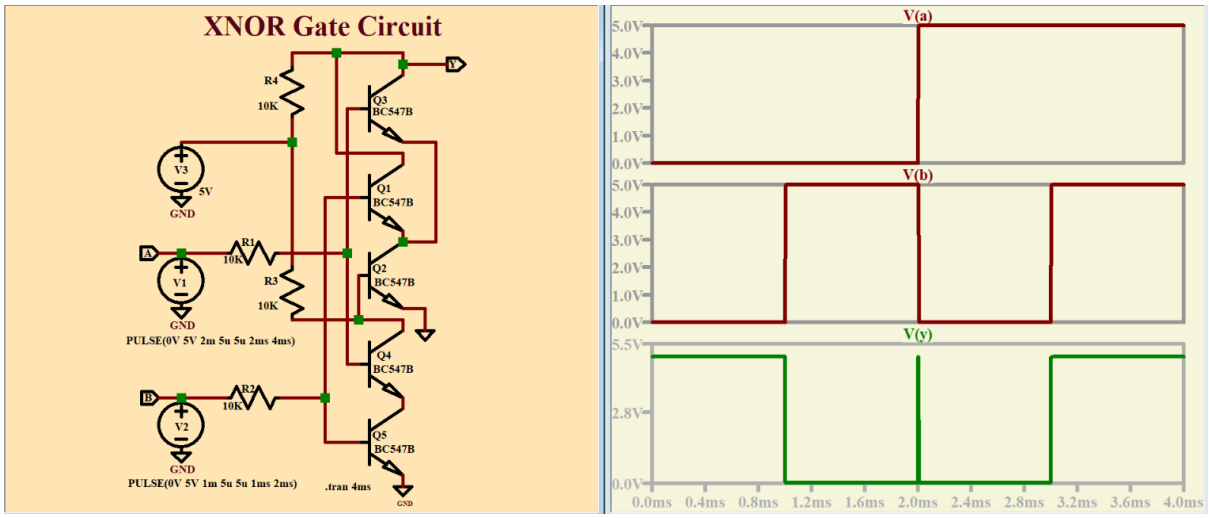
$$Y = AB + A`B`$$

$$Or\ Y = A \Theta B$$

Truth Table

Input (A)	Input (B)	Output [Y = A Θ B]
0	0	1
0	1	0
1	0	0
1	1	1

LTspice Simulation



Physical Circuit



This was a good reminder that sometimes a circuit that doesn't produce the output I initially expected isn't necessarily useless.

It may simply be performing another valid logic function.

The Seven Gates

After completing the circuits, the final set was -

Gate	Boolean Expression	Output becomes High when...
NOT	A`	Input is LOW
NOR	(A + B)`	Both inputs are LOW
OR	A + B	Any input is HIGH
NAND	(A.B)`	At least one input is LOW
AND	A.B	Both inputs are HIGH
XNOR	AB + A`B` Or Y = A Θ B	Inputs are the same
XOR	AB` + A`B Or Y = A ⊗ B	Inputs are different

The interesting part for me was that these were no longer just seven symbols in a digital-electronics textbook.

I could point to an actual transistor and say -

"This transistor is responsible for this switching behaviour."

Simulation vs Real Circuit

One thing I noticed during the project is that the LTspice output doesn't necessarily have to be exactly -

0 V or 5V

Because, the **BC547B isn't an ideal switch.**

For example, depending on the circuit, the HIGH output can be somewhat below 5V.

That's not automatically a failure.

The important question is -

Does the circuit produce the correct logical state?

This was another reason I preferred checking the waveform against the truth table instead of judging the circuit only by whether the output reached exactly 5V.

What This Project Actually Taught Me

This project taught me something that I don't think I would have understood as clearly just by studying the gate symbols.

Before this, I could look at -

Y=A+B

And know this represents **OR.**

Now I can also look at the transistor circuit and think -

- Which transistor is ON?
- Which one is OFF?
- Where is the output being pulled?
- Why is the output HIGH here?
- Why does it become LOW when the input changes?

That connection between **Boolean logic and actual transistor switching** was the main thing I wanted from this project.

The Bigger Connection

For me, the project ended up connecting several things I had studied separately -

Boolean Algebra

↓

Truth Table

↓

Logic Gate

↓

Transistor Switching

↓

LTspice

↓

Physical Circuit

And that's probably the biggest takeaway from the project.

The symbol is just the representation.

The transistor circuit is where the actual electronics happens.

Conclusion

So, technically, this project is about implementing logic gates using **BC547 transistors.**

But for me, that's not really the whole story.

It started with -

"Why did I lose marks for using the gate symbol?"

Then became -

"Okay, so what does the actual circuit look like?"

And eventually -

"I have BC547s anyway... why not build it?"

From there, I simulated the circuits, checked the truth tables, debugged the unexpected outputs and implemented the same concepts using physical components.

What started as an exam mistake ended up becoming a practical way of understanding something I had previously only seen as symbols and Boolean equations.

And honestly, that's probably the best part of the project.